

ТЕХНИЧЕСКОЕ ЗАДАНИЕ

Разработчик: Жусупекова Сезим Саламатовна

MVP 3D PvP-игры для Telegram Mini Apps

1. Назначение документа

1.1 Цель документа

Настоящее техническое задание (далее — **ТЗ**) предназначено для **формализации и фиксации всех технических, функциональных и архитектурных требований** к разработке MVP версии 3D PvP-игры для платформы **Telegram Mini Apps**.

Документ используется как:

- основной источник требований для разработки;
- база для оценки сроков и объёма работ;
- критерий приёмки результата (Definition of Done);
- основание для разрешения спорных ситуаций между заказчиком и исполнителем.

Любая функциональность, **не описанная в данном ТЗ**, считается **не входящей в объём MVP**, если иное не согласовано письменно.

1.2 Назначение продукта

Разрабатываемый продукт представляет собой **онлайн 3D-мини-игру**, запускаемую внутри Telegram Mini Apps, с поддержкой **матчей на 4 игроков в реальном времени**.

Игра предназначена для:

- быстрого запуска пользователем без установки приложений;
- коротких игровых сессий;
- демонстрации core-геймплея и сетевой архитектуры;
- последующего расширения (визуал, режимы, монетизация).

1.3 Назначение MVP

MVP (Minimum Viable Product) создаётся с целью:

1. Проверки:
 - работоспособности real-time взаимодействия в Telegram WebView;
 - стабильности WebGL (Three.js) на мобильных устройствах;
 - корректности кастомной плоской физики.
2. Демонстрации:
 - базового игрового процесса;
 - сетевой архитектуры server-authoritative;
 - возможности подключения 4 игроков в одну сессию.
3. Подготовки основы:
 - для масштабирования проекта;
 - для добавления анимаций, эффектов и игровых режимов;

- для дальнейшей коммерческой разработки.

1.4 Ограничения MVP

В рамках данного ТЗ **осознанно вводятся ограничения**, направленные на снижение сложности и сроков разработки.

В MVP:

- отсутствуют сложные физические движки;
- отсутствует сложная анимация персонажей;
- отсутствует прогрессия, аккаунты и сохранения;
- отсутствует матчмейкинг и лобби;
- отсутствуют визуальные пост-эффекты (Bloom, SSAO и т.п.).

Все ограничения являются **частью проектного решения**, а не упущением.

1.5 Юридический и договорной статус документа

Данное ТЗ:

- является техническим описанием объёма работ;
- используется для приёмки результата;
- не описывает коммерческие условия, сроки оплаты и авторские права (оформляются отдельно);
- может быть дополнено приложениями (диаграммы, схемы, референсы).

1.6 Термины и сокращения

Термин	Значение
MVP	Минимально жизнеспособная версия продукта
TMA	Telegram Mini Apps
WebView	Встроенный браузер Telegram
Server-authoritative	Сервер является единственным источником истины
Tick	Один шаг серверного обновления
Snapshot	Состояние игры, отправляемое клиентам

2. Область работ (Scope)

2.1 Общие положения

В рамках настоящего ТЗ исполнитель обязуется разработать **рабочий MVP 3D PvP-игры для Telegram Mini Apps**, предназначенной для запуска внутри Telegram WebView и ориентированной на мобильные устройства.

MVP должен представлять собой **играбельную версию игры**, в которой 4 игрока могут подключиться к одному матчу в реальном времени и взаимодействовать друг с другом через базовую игровую механику.

Нижеуказанная стоимость этапа является **условной**.

Любые работы, не описанные в настоящем разделе, **не входят в объём MVP**.

2.2 Работы, входящие в Scope MVP

В объём работ по MVP **обязательно** входят следующие задачи.

2.2.1 Локация (арена)

- Создание и интеграция **статичной 3D-локации** в виде четырёхугольного ринга.
- Локация должна быть **оформлена в едином визуальном стиле игры**, соответствующем референсам или описанию заказчика.
- Арена ограничивает игровое пространство и используется для расчёта физики мяча.
- Локация оптимизирована под Telegram WebView (mobile-first).

2.2.2 Персонажи

- Реализация **4 разных персонажей** (визуально различимых).
- Каждый персонаж:
 - закреплён за своей стороной ринга;
 - может двигаться вперёд / назад / влево / вправо в пределах своей стороны;
 - **не вращается полностью** при движении (ориентация модели фиксирована).
- Персонажи интегрированы в 3D-сцену и синхронизируются в real-time.

2.2.3 Мяч и базовая физика

- Реализация игрового мяча:
 - одноцветный;
 - без мелких узоров и сложных элементов;
 - с небольшой визуальной тенью.
- Мяч:
 - перемещается **только по плоскости** (без прыжков);
 - отражается от игроков;
 - отражается от стен и углов ринга;
 - меняет траекторию в зависимости от угла удара;
 - изменяет скорость в зависимости от типа отражения;
 - взаимодействует с магнитными зонами (описаны отдельно).
- Физика мяча реализуется как **лёгкая кастомная физика**, без использования тяжёлых движков.

2.2.4 Онлайн и real-time

- Реализация **онлайн-режима на 4 игроков**.
- Используется real-time синхронизация (WebSocket).
- Сервер:
 - авторитетен по физике и состоянию игры;
 - обрабатывает ввод игроков;
 - рассылает состояние клиентам.
- Клиенты:
 - отправляют только ввод;
 - получают и отображают состояние игры.

2.2.5 Интеграция с Telegram Mini Apps

- Игра запускается внутри Telegram Mini Apps.
- Учитываются ограничения Telegram WebView.

- Интерфейс и управление ориентированы на **мобильные устройства**.

2.3 Работы, НЕ входящие в Scope MVP

В рамках MVP **не входят**:

- дополнительные игровые режимы;
- расширенные правила, рейтинги, прогрессия;
- сложные визуальные эффекты и анимации;
- монетизация и мета-игровые системы.

Все перечисленные пункты могут быть реализованы **только в рамках отдельного этапа после MVP**.

2.4 Управление изменениями Scope

Любое расширение Scope:

- оформляется отдельным соглашением;
- влияет на сроки и стоимость;
- не считается обязательным в рамках данного ТЗ.

2.5 Фиксация объёма MVP и ограничений функциональности

Назначение MVP

В рамках настоящего ТЗ разрабатывается **исключительно MVP (Minimum Viable Product)** — минимально необходимая, полностью работоспособная версия игры, достаточная для запуска, тестирования и демонстрации основного геймплея.

MVP предназначен для:

- проверки работоспособности игры в Telegram Mini Apps;
- демонстрации real-time взаимодействия 4 игроков;
- тестирования базовой 3D-сцены, управления и физики мяча.

Состав MVP (фиксированный)

В MVP **обязательно входят только следующие элементы**:

- одна статичная 3D-локация (четырёхугольный ринг);
- 4 персонажа, закреплённые за сторонами ринга;
- управление персонажами (вперёд / назад / влево / вправо) без полного вращения моделей;
- игровой мяч с плоской кастомной физикой:
 - движение по плоскости;
 - отражение от игроков, стен и углов;
 - изменение траектории от угла удара;
 - изменение скорости;
 - взаимодействие с магнитными зонами;
- онлайн-режим на 4 игроков в реальном времени;
- запуск игры внутри Telegram Mini Apps.

Ограничения MVP (обязательные)

В рамках MVP **не реализуются и не считаются частью работ**:

- дополнительные игровые режимы;

- альтернативные карты или арены;
- расширенные правила, рейтинги, турниры;
- прогрессия, аккаунты, сохранения;
- визуальные эффекты высокого уровня;
- сложные анимации персонажей;
- расширенный UI и мета-интерфейс.

Любые доработки, расширения или дополнительные механики **не входят в MVP** и могут быть реализованы **только в рамках отдельного этапа разработки** по отдельному соглашению.

Критерий интерпретации

При любых разночтениях или спорных ситуациях приоритет имеет следующая трактовка:

Если функциональность явно не указана в составе MVP — она считается не входящей в объём работ.

2.6 2.X Ограничения MVP и условия выполнения работ

2.6.1 Ассеты и визуальные материалы

В рамках MVP используются 3D-ассеты (арена, персонажи, мяч), необходимые для демонстрации базового игрового процесса.

- Если заказчик **предоставляет готовые ассеты или референсы**, исполнитель использует их при реализации.
- Если заказчик **не предоставляет готовые ассеты**, исполнитель разрабатывает **упрощённые low-poly ассеты**, достаточные для реализации MVP и соответствующие стилю игры.
- Художественная детализация, высокополигональные модели, сложные материалы и визуальные улучшения **не входят в объём MVP**, если не согласованы отдельно.

2.6.2 Сервер и инфраструктура

В рамках MVP исполнитель:

- реализует серверную часть игры (WebSocket-сервер);
- передаёт исходный код сервера и инструкцию по запуску;
- обеспечивает корректную работу сервера в локальной или тестовой среде.

Размещение сервера, оплата хостинга, домены и эксплуатация инфраструктуры **осуществляются заказчиком**, если иное не согласовано отдельно.

Поддержка, администрирование и мониторинг сервера после сдачи MVP **не входят в объём работ**.

2.6.3 Поддержка после сдачи MVP

После сдачи и приёмки MVP исполнитель исправляет **только критические ошибки**, которые:

- препятствуют запуску игры;
- делают игру неиграбельной;
- приводят к крашу клиента или сервера.

Все остальные изменения, улучшения, оптимизации, правки визуала и расширение функциональности **не входят в MVP** и выполняются **только в рамках отдельного этапа разработки**.

3. Технический стек и требования к среде выполнения

3.1 Общие принципы

Разрабатываемый продукт является **Telegram Mini App**, запускаемой внутри встроенного **Telegram WebView**, в связи с чем все технические решения должны быть ориентированы на:

- мобильные устройства (iOS / Android);
- ограниченные ресурсы WebView (CPU, GPU, память);
- быстрый запуск без установки приложений;
- стабильную работу WebGL внутри Telegram.

Разработка ведётся по принципу **mobile-first**, при котором мобильные устройства являются основной целевой платформой, а desktop-версия рассматривается как вторичная.

Использование альтернативных технологий и отклонение от указанного стека **не допускается без предварительного согласования**, так как может привести к нестабильной работе внутри Telegram.

3.2 Клиентская часть (Frontend)

3.2.1 Язык и среда выполнения

- Язык разработки: **JavaScript** или **TypeScript** (TypeScript предпочтителен).
- Среда выполнения: **Telegram WebView**.
- Поддерживаемые платформы:
 - Telegram iOS;
 - Telegram Android;
 - Telegram Desktop (опционально, без приоритетной оптимизации).

3.2.2 Framework и структура клиента

- Framework: **Next.js**.
- Страница игры:
 - путь: `/game`;
 - загрузка сцены через `dynamic import`;
 - Server-Side Rendering (SSR) — **отключён**.
- Клиентская сборка:
 - production-режим;
 - минимизированный и оптимизированный бандл;
 - отсутствие debug-зависимостей в финальной версии.

3.2.3 Рендеринг и графика

- Библиотека рендеринга: **Three.js**.
- Используемый рендерер: `WebGLRenderer`.
- Сцена включает:
 - `Scene`;
 - `PerspectiveCamera`;

- DirectionalLight (1 шт.);
 - AmbientLight (1 шт.).
- Камера:
 - фиксированная;
 - обеспечивает полный обзор арены;
 - не требует управления пользователем.
- Запрещено:
 - post-processing (Bloom, SSAO, Motion Blur и т.п.);
 - HDR-рендеринг;
 - сложные кастомные шейдеры.

3.2.4 Управление ресурсами и ассетами

- Формат всех 3D-ассетов: **GLB (glTF binary)**.
- Загрузка осуществляется через GLTFLoader.
- Требования к ассетам:
 - единая система измерений (1 unit = 1 метр);
 - корректные pivot (0,0,0);
 - отсутствие скрытых, неиспользуемых или фоновых мешей;
 - минимальное количество материалов и текстур;
 - оптимизированный полигонаж для мобильных устройств.

3.3 Серверная часть (Backend)

3.3.1 Язык и платформа

- Язык разработки: **JavaScript**.
- Платформа: **Node.js версии 18 или выше**.
- Архитектура:
 - один серверный процесс;
 - без кластеризации и шардинга;
 - без сохранения состояния между перезапусками.

3.3.2 WebSocket-взаимодействие

- Протокол: **WebSocket**.
- Библиотека: **ws**.
- Сервер обязан:
 - принимать и закрывать подключения;
 - обрабатывать реконнекты;
 - поддерживать одно игровое пространство (комнату) на 4 игроков.

3.3.3 Ограничения серверной логики

- Максимальное количество игроков в комнате: **4**.
- Попытка подключения 5-го клиента должна завершаться отказом.
- Сервер является **единственным источником истины**:
 - физика;
 - позиции объектов;
 - состояние матча.
- Клиентские данные проходят валидацию.

3.4 Интеграция с Telegram Mini Apps

3.4.1 Используемый SDK

- Используется официальный Telegram Mini Apps SDK:
 - telegram-web-app.js.
- Клиент обязан получать:
 - Telegram.WebApp.initDataUnsafe.user.id;
 - Telegram.WebApp.initDataUnsafe.user.username.

3.4.2 Ограничения Telegram WebView

- Ограниченные возможности GPU по сравнению с браузером.
- Возможны ограничения FPS.
- Поведение WebGL может отличаться от desktop-браузеров.
- Запрещено использование системных API устройства.

3.5 Mobile-first требования (обязательные)

3.5.1 Экран и масштабирование

- Игра ориентирована на **экраны смартфонов**.
- WebGL-канвас должен:
 - занимать максимально доступную область экрана;
 - корректно масштабироваться под размер WebView;
 - пересчитываться при resize.

3.5.2 Ориентация устройства

- Основная поддерживаемая ориентация:
 - **Portrait (вертикальная)** либо
 - **Landscape (горизонтальная)** — выбирается на этапе реализации.
- При смене ориентации:
 - камера пересчитывается;
 - канвас корректно масштабируется;
 - UI не ломается.

3.5.3 Управление на мобильных устройствах

- Обязательное наличие тач-управления:
 - экранный стик либо
 - D-pad (стрелки).
- Управление должно:
 - поддерживать удержание;
 - быть устойчивым к мультитачу;
 - не требовать высокой точности касаний.
- Минимальный размер интерактивной зоны: **44×44 px**.

3.5.4 Производительность

- Целевой FPS в Telegram WebView:
 - **средний ≥ 45 FPS**.
- Запрещены тяжёлые визуальные эффекты.
- Тени:
 - облегчённые;
 - blob-тень для мяча;
 - минимум динамических теней.

3.6 Ограничения на сторонние библиотеки

Допускается использование:

- Three.js;
- ws;
- лёгких вспомогательных библиотек (math, helpers).

Запрещено использование:

- физических движков (Ammo.js, Cannon.js, Rapier и аналоги);
- тяжёлых UI-фреймворков;
- экспериментальных WebGL-библиотек.

4. Архитектура системы и взаимодействие компонентов

4.1 Общая архитектурная модель

Архитектура игры строится по принципу **client–server** с использованием **server-authoritative** подхода.

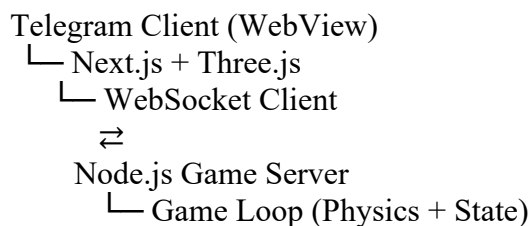
Сервер является **единственным источником истины** для:

- состояния матча;
- позиций игроков;
- физики мяча;
- обработки столкновений.

Клиенты не выполняют расчётов, влияющих на итоговое состояние игры, и используются исключительно для:

- отправки пользовательского ввода;
- визуализации полученного состояния;
- локальной интерполяции для сглаживания.

4.2 Высокоуровневая схема взаимодействия



4.3 Клиентская архитектура

4.3.1 Основные модули клиента

Клиентская часть должна быть логически разделена на следующие модули:

1. **App / Bootstrap**
 - инициализация Telegram Mini App;
 - проверка среды запуска;
 - инициализация WebSocket-подключения.
2. **Render Module**
 - инициализация Three.js сцены;
 - управление камерой и светом;
 - рендер-цикл.
3. **Assets Loader**

- загрузка GLB-ассетов;
- контроль прогресса загрузки;
- обработка ошибок.
- 4. **Input Controller**
 - обработка клавиатуры;
 - обработка тач-ввода;
 - формирование input-команд.
- 5. **Network Client**
 - отправка input-сообщений;
 - приём снапшотов;
 - обработка реконнекта.
- 6. **State Interpolator**
 - сглаживание позиций игроков;
 - сглаживание движения мяча;
 - компенсация сетевой задержки.
- 7. **UI / HUD Layer**
 - отображение никнеймов;
 - сетевой статус;
 - подсказки управления.

4.4 Серверная архитектура

4.4.1 Основные модули сервера

Сервер должен быть реализован с логическим разделением на следующие компоненты:

1. **WebSocket Gateway**
 - приём и закрытие подключений;
 - обработка handshake;
 - идентификация клиентов.
2. **Room Manager**
 - создание игровой комнаты;
 - назначение сторон арены;
 - контроль количества игроков (макс. 4).
3. **Input Processor**
 - приём и валидация input-сообщений;
 - нормализация входных данных.
4. **Game Loop**
 - фиксированный тик (60 Hz);
 - обновление состояния игроков;
 - расчёт физики мяча;
 - обработка коллизий.
5. **State Broadcaster**
 - формирование снапшотов;
 - рассылка состояния всем клиентам;
 - контроль частоты отправки.
6. **Connection Monitor**
 - обработка дисконнектов;
 - реконнект клиентов;
 - очистка состояния при выходе игрока.

4.5 Игровой цикл (Game Loop)

4.5.1 Частоты

- Серверный тик: **60 раз в секунду**.
- Частота отправки снапшотов:
 - минимум 20 Hz;
 - максимум 60 Hz (в зависимости от нагрузки).

4.5.2 Порядок выполнения тика

На каждом серверном тике выполняются строго в следующем порядке:

1. Чтение накопленного ввода игроков.
2. Обновление позиций игроков.
3. Расчёт движения мяча.
4. Проверка столкновений:
 - мяч ↔ игрок;
 - мяч ↔ борта;
 - мяч ↔ углы арены.
5. Применение изменений скорости и направления мяча.
6. Формирование актуального состояния матча.
7. Отправка снапшота клиентам.

4.6 Сетевая синхронизация

4.6.1 Типы данных

Передаваемые данные делятся на:

- **Input** — от клиента к серверу;
- **Snapshot** — от сервера к клиенту.

Клиенты **не отправляют**:

- позиции объектов;
- скорости;
- данные физики.

4.6.2 Интерполяция на клиенте

Клиент обязан:

- хранить несколько последних снапшотов;
- интерполировать позиции между ними;
- избегать телепортаций и резких скачков.

Экстраполяция допускается **только кратковременно** при потере пакетов.

4.7 Обработка ошибок и устойчивость

4.7.1 Потеря соединения

- При разрыве соединения:
 - клиент отображает статус “Reconnecting”;
 - выполняется автоматическая попытка переподключения.
- При успешном реконнекте:
 - сервер восстанавливает состояние игрока;
 - игрок возвращается на свою сторону.

4.7.2 Некорректные данные

- Сервер обязан:
 - валидировать входные данные;
 - игнорировать некорректный ввод;
 - не допускать краша процесса.

4.8 Ограничения архитектуры MVP

В рамках MVP:

- используется одна игровая комната;
- отсутствует масштабирование по горизонтали;
- отсутствует балансировка нагрузки;
- отсутствует сохранение состояния.

Все ограничения являются **осознанными** и направлены на ускорение разработки MVP.

5. Игровая сцена, система координат и правила размещения объектов

5.1 Назначение игровой сцены

Игровая сцена представляет собой **статичную 3D-локацию (арену)**, внутри которой происходит весь игровой процесс.

Сцена служит:

- пространственным ограничением игры;
- базой для расчёта физики мяча;
- системой координат для позиционирования игроков.

Сцена не меняется во время матча и не содержит динамически создаваемых элементов, кроме игроков и мяча.

5.2 Система координат

5.2.1 Оси координат

Используется стандартная правая система координат Three.js:

- **Ось X** — горизонталь (лево ↔ право);
- **Ось Z** — глубина (вперёд ↔ назад);
- **Ось Y** — высота (используется минимально).

Все игровые расчёты выполняются **в плоскости XZ**.

Координата Y используется только для визуального отображения (высота объектов и тень).

5.2.2 Нулевая точка сцены

- Центр арены располагается в точке **(0, 0, 0)**.

- Pivot модели арены обязан совпадать с (0,0,0).
- Все остальные объекты позиционируются **относительно центра арены**.

5.3 Арена (Ring)

5.3.1 Геометрические параметры

Арена представляет собой прямоугольник со следующими параметрами:

Параметр	Значение
Ширина (X)	16.0
Глубина (Z)	10.0
Высота борта (Y)	0.5
Толщина борта	0.3

Границы арены:

- $X \in [-8.0 ; 8.0]$
- $Z \in [-5.0 ; 5.0]$

5.3.2 Ограничения движения

- Мяч не может покинуть пределы арены.
- Игроки не могут покидать пределы своих зон.
- Все столкновения с бортами обрабатываются сервером.

5.4 Деление арены на игровые стороны

Арена логически делится на **4 стороны**, каждая из которых закрепляется за одним игроком.

5.4.1 Зоны сторон

Сторона	Ограничение
North	$Z \in [2.5 ; 5.0]$
South	$Z \in [-5.0 ; -2.5]$
West	$X \in [-8.0 ; -4.0]$
East	$X \in [4.0 ; 8.0]$

Игрок жёстко ограничен координатами своей зоны и не может пересекать центральную область арены.

5.5 Размещение игроков

5.5.1 Начальные позиции

- Каждый игрок появляется **в центре своей стороны**.
- Начальная позиция задаётся сервером при подключении.

Пример:

- North: (0, 0, 3.75)
- South: (0, 0, -3.75)
- West: (-6.0, 0, 0)
- East: (6.0, 0, 0)

5.5.2 Ограничения перемещения

- Перемещение возможно только в пределах зоны стороны.
- Выход за пределы зоны жёстко запрещён и корректируется сервером.
- Клиент может дополнительно ограничивать движение локально (визуально).

5.6 Размещение мяча

5.6.1 Начальная позиция

- Мяч появляется в центре арены:
 - (0, 0, 0)
- Высота мяча:
 - $Y = 0.5$ (визуально)
- Физически мяч считается находящимся в плоскости XZ.

5.6.2 Ограничения

- Мяч не выходит за пределы арены.
- Столкновения с бортами и углами обрабатываются сервером.
- Визуальное положение мяча синхронизировано с серверным состоянием.

5.7 Камера

5.7.1 Тип камеры

- Тип: PerspectiveCamera
- Камера фиксированная
- Камера не управляется игроком

5.7.2 Требования к обзору

- Камера должна охватывать всю арену целиком.
- Все игроки и мяч должны быть видимы одновременно.
- Камера не должна искажать пропорции объектов при разных разрешениях экрана.

5.8 Освещение сцены

5.8.1 Источники света

- 1 × DirectionalLight
- 1 × AmbientLight

5.8.2 Ограничения

- Минимальное количество теней.
- Без динамических смен освещения.
- Без пост-обработки.

5.9 Ограничения сцены MVP

В рамках MVP:

- сцена полностью статична;
- отсутствует разрушаемость;
- отсутствуют динамические декорации;
- отсутствуют погодные и визуальные эффекты.

6. Игровые персонажи (Player Entity)

6.1 Назначение сущности игрока

Игровой персонаж (далее — **Player**) — управляемый объект, который:

- перемещается **только в пределах своей стороны арены**;
- может **отбивать мяч** при столкновении;
- отображается в 3D-сцене как модель персонажа + (опционально) тележка/платформа;
- синхронизируется сервером (позиция авторитетна у сервера).

6.2 Количество персонажей и визуальные варианты

- В матче участвуют **ровно 4 игрока**.
- Каждый игрок использует отдельный визуальный скин/модель (MVP допускает упрощение):
 - кот
 - утка
 - голубь
 - собака

Допускаются 2 варианта поставки ассетов:

1. один player.glb + смена материалов/частей,
2. отдельные файлы cat.glb, duck.glb, pigeon.glb, dog.glb.

Требование MVP: уникальность должна быть визуально различима (форма/цвет/силуэт).

6.3 Требования к 3D-модели персонажа

6.3.1 Формат и структура

- Формат: **GLB**
- Pivot: **(0,0,0)** в основании модели (или строго согласованно по центру коллайдера)
- Ориентация: модель направлена вдоль **+Z** (единый стандарт)
- Количество материалов: минимально (желательно 1–2)
- Текстуры: по возможности отсутствуют, либо минимальные (малый вес)

6.3.2 Масштаб (ориентир)

- ширина по X: ~ 2.2
- глубина по Z: ~ 0.7
- высота по Y: ~ 2.0

Важно: фактический визуальный размер может отличаться, но **коллайдер и физические параметры должны соответствовать заданным размерам**, чтобы поведение было одинаковым для всех персонажей.

6.4 Ограничение на вращение

Ключевое требование:

Персонаж **не должен вращаться** вслед за направлением движения.

- Модель остаётся в фиксированной ориентации (например, всегда “смотрит” в +Z).
- Допускается только:
 - смена анимации (если есть),
 - смена позы/эффекта,
 - но не поворот по Y.

Это требование относится как к визуалу, так и к логике: **yaw rotation = 0** всегда (либо фиксированное значение).

6.5 Управление и движение персонажа

6.5.1 Тип движения

- Движение линейное, без инерции и ускорения.
- Игрок перемещается по плоскости XZ.

6.5.2 Скорость

- Базовая скорость: **6.0 units/sec**
- Диапазон допустимых скоростей (настройка): 4.0–8.0

6.5.3 Ввод

Игрок управляет персонажем через:

- Desktop: W/A/S/D (или стрелки)
- Mobile: виртуальный стик или D-pad

На сервер передаётся не позиция, а **вектор ввода**:

- $\text{inputX} \in [-1, 1]$
- $\text{inputZ} \in [-1, 1]$
- Нормализация диагонали обязательна (чтобы диагональ не была быстрее).

6.6 Ограничение движения по стороне (Side Lock)

6.6.1 Правило

Игрок может перемещаться **только внутри своей зоны** (см. пункт 5.4).

Сервер обязан на каждом тике:

- проверять позицию игрока;
- “клампить” координаты в допустимый диапазон.

6.6.2 Диапазоны (фиксированные для MVP)

- North: $Z \in [2.5 ; 5.0]$, $X \in [-8.0 ; 8.0]$
- South: $Z \in [-5.0 ; -2.5]$, $X \in [-8.0 ; 8.0]$
- West: $X \in [-8.0 ; -4.0]$, $Z \in [-5.0 ; 5.0]$
- East: $X \in [4.0 ; 8.0]$, $Z \in [-5.0 ; 5.0]$

6.7 Коллизия персонажа (физическое представление)

6.7.1 Тип коллайдера

Для MVP используется упрощённый коллайдер:

Вариант А (предпочтительный): AABB прямоугольник

- размер по X: **2.2**
- размер по Z: **0.7**

Вариант Б: Capsule 2D (если удобнее математически)

- ширина 2.2, глубина 0.7 условно через радиус/длину

Требование: тип коллайдера фиксируется и используется одинаково для всех игроков.

6.8 Отбивание мяча (удар)

6.8.1 Условие удара

Удар считается, когда:

- происходит столкновение “мяч $\leftrightarrow$ коллайдер игрока”
- и мяч движется в сторону игрока или пересекает его зону контакта.

6.8.2 Параметры удара (задаются сервером)

- Игрок влияет на траекторию мяча:
 - через нормаль столкновения,
 - через свой текущий вектор движения.

Минимальная формула (концептуально):

- отражение вектора скорости от поверхности игрока
- добавление импульса от движения игрока

6.9 Идентификация игрока и данные профиля

При подключении клиент отправляет (или сервер получает из Telegram initData):

- telegramUserId
- username (или fallback “Player ###”)

Сервер назначает:

- playerId (внутренний id)
- side (north/south/west/east)

6.10 Визуальные элементы игрока (MVP)

Обязательные элементы:

- 3D-модель персонажа
- никнейм над персонажем (billboard / sprite text)

Опционально (если не мешает производительности):

- лёгкая тень под персонажем (можно отключить в MVP)

6.11 Требования к синхронизации игрока

- Позиция игрока авторитетна на сервере.
- Клиент:
 - отображает позицию через интерполяцию,
 - не “телепортирует” игрока без необходимости,
 - корректно отображает задержку без рывков.

6.12 Критерии готовности пункта (Acceptance Criteria)

Персонажи считаются реализованными, если:

1. В матче видны 4 игрока, каждый на своей стороне.
2. Игрок перемещается W/A/S/D и через тач-контрол.
3. Игрок **не вращается** при движении.
4. Игрок не может выйти за границы своей зоны.
5. Никнейм отображается над игроком.
6. При столкновении с мячом сервер корректно фиксирует “удар” (отражение).

7. Мяч и кастомная плоская физика (Ball Entity)

7.1 Назначение сущности мяча

Мяч (далее — **Ball**) является основным динамическим объектом матча и должен:

- двигаться **строго по плоскости XZ** (без прыжков);
- отражаться от игроков, бортов и углов арены;
- менять траекторию в зависимости от угла удара и движения игрока;
- изменять скорость в зависимости от типа отражения (удар/угол/магнитная зона);
- иметь **лёгкую тень** под собой (blob-shadow);
- быть полностью **server-authoritative** (вся физика на сервере).

7.2 Требования к 3D-модели мяча

7.2.1 Формат и параметры

- Формат: **ball.glb**
- Pivot: **центр мяча** в точке (0,0,0)

- Радиус (визуальный): **0.5 units** (допустимо 0.45–0.55)
- Материал:
 - однотонный (светло-серый / голубой);
 - без мелких узоров и деталей;
 - простой блик допускается (не PBR-сложный).
- Полигональность: low-poly / средняя, без излишней детализации.

7.2.2 Размещение в сцене

- Визуальная высота: $Y = 0.5$ (равна радиусу)
- Игровая физика идёт в 2D (XZ), компонент Y в расчётах не участвует.

7.3 Физическая модель (2D, без движков)

7.3.1 Состояние мяча (server state)

Сервер хранит состояние мяча:

- позиция: $\text{pos} = \{ x, z \}$
- скорость (вектор): $\text{vel} = \{ x, z \}$

Дополнительно (по необходимости):

- $\text{speed} = \text{length}(\text{vel})$
- lastHitByPlayerId (кто последним отбил)
- lastHitTime (анти-дабл-хит/дребезг)

7.3.2 Параметры физики (фиксированные для MVP)

Параметр	Значение	Комментарий
Server Tick	60 Hz	фиксированный шаг
minSpeed	2.0	минимальная скорость
baseSpeed	6.0	стартовая скорость после старта
maxSpeed	12.0	верхний предел скорости
linearDamping	0.995 / tick	лёгкое затухание
restitutionWall	1.0	отражение от борта без потери
restitutionPlayer	1.0	базово без потери (скорость меняется импульсом)
hitImpulse	2.5	добавка от удара (настройка)
angleInfluence	1.0	влияние точки удара на угол

Все параметры должны быть вынесены в конфиг и легко настраиваться.

7.4 Движение мяча

На каждом server-tick:

1. Применить затухание:
 - $\text{vel} *= \text{linearDamping}$
2. Ограничить скорость:
 - если $|\text{vel}| < \text{minSpeed} \rightarrow$ нормализовать и поставить minSpeed
 - если $|\text{vel}| > \text{maxSpeed} \rightarrow$ нормализовать и поставить maxSpeed

3. Обновить позицию:

- $pos += vel * dt$

Где $dt = 1/60$.

7.5 Столкновение мяча со стенами (борта арены)

7.5.1 Границы арены

Границы физики мяча (учитывая радиус):

- $minX = -8.0 + radius$
- $maxX = 8.0 - radius$
- $minZ = -5.0 + radius$
- $maxZ = 5.0 - radius$

7.5.2 Правила отражения

Если мяч пересёк границу:

- для вертикальных стен (по X):
 - инвертировать $vel.x = -vel.x$
 - “вытолкнуть” позицию внутрь ($clamp pos.x$)
- для горизонтальных стен (по Z):
 - инвертировать $vel.z = -vel.z$
 - “вытолкнуть” позицию внутрь ($clamp pos.z$)

Дополнительно (важно для стабильности):

- если мяч “залип” в стене, повторно клампить позицию и не допускать многократных инверсий в одном тике.

7.6 Столкновение мяча с углами

7.6.1 Проблема углов

В углах возможны “застревания” и некорректное многократное отражение.

7.6.2 Требования

- При попадании в угол (контакт одновременно с X и Z границей):
 - инвертировать обе компоненты:
 - $vel.x = -vel.x$
 - $vel.z = -vel.z$
 - после отражения выставить позицию внутрь допустимых границ.
- Запрещено, чтобы мяч оставался неподвижным в углу.
- Допускается добавление минимального “разброса” (epsilon) к направлению, если выявлено залипание (но в MVP можно без рандома, если стабильно).

7.7 Столкновение мяча с игроком (удар)

7.7.1 Представление коллизии

- Мяч: окружность радиуса $radius$
- Игрок: прямоугольник (AABB) с размерами:
 - $widthX = 2.2$

- $depthZ = 0.7$

Коллизия определяется как пересечение “круга” с прямоугольником в плоскости XZ.

7.7.2 Расчёт нормали удара

Нормаль удара должна определяться от точки ближайшего контакта (или по направлению от игрока к мячу):

- $n = \text{normalize}(\text{ballPos} - \text{closestPointOnPlayerAABB})$

7.7.3 Базовое отражение (рефлексия)

Скорость отражается от нормали:

- $\text{vel} = \text{reflect}(\text{vel}, n)$

Где $\text{reflect}(v, n) = v - 2 * \text{dot}(v, n) * n$

7.7.4 Влияние “угла атаки”

Угол отражения должен зависеть от точки удара:

- если мяч попадает ближе к краям игрока по X (для North/South сторон) — траектория уходит сильнее вбок;
- если ближе к краям по Z (для West/East сторон) — аналогично.

Требование MVP:

точка удара должна ощутимо влиять на траекторию, чтобы игрок мог “прицеливаться”.

Практическая реализация:

- добавить к нормали небольшой сдвиг, зависящий от смещения контакта относительно центра игрока, и затем нормализовать.

7.7.5 Импульс от движения игрока

Если игрок двигался в момент удара, мяч должен получить добавочный импульс:

- $\text{vel} += \text{playerMoveDir} * \text{hitImpulse}$

Где playerMoveDir — нормализованный вектор движения игрока.

Условия:

- импульс применяется только если игрок реально двигался ($|\text{input}| > \text{threshold}$).
- итоговая скорость ограничивается maxSpeed .

7.7.6 Анти-дребезг (двойные удары)

Чтобы мяч не “дрожал” внутри коллайдера игрока:

- после удара вводится кулдаун контакта:
 - $\text{hitCooldown} = 80\text{--}120 \text{ ms}$ (настройка)

- пока не истёк cooldown, повторное столкновение с тем же игроком игнорируется, если мяч ещё в зоне контакта.

7.8 Магнитные зоны / поля (обязательная часть MVP)

7.8.1 Общие требования

В рамках MVP в игре **обязательно должны быть реализованы магнитные зоны (поля)**, влияющие на движение игрового мяча, как указано в описании проекта заказчика.

Магнитные зоны являются частью базовой механики мяча и **входят в обязательный объём MVP**.

Исключение или упрощение магнитных зон без согласования с заказчиком **не допускается**.

7.8.2 Назначение магнитных зон

Магнитные зоны предназначены для:

- изменения траектории движения мяча;
- усложнения и разнообразия PvP-взаимодействия;
- создания дополнительных тактических возможностей для игроков.

Магнитные зоны воздействуют **только на мяч** и **не влияют на игроков**.

7.8.3 Минимальные требования к магнитным зонам (MVP)

В рамках MVP должно быть реализовано:

- **Количество зон:**
минимум **4 магнитные зоны**, расположенные в угловых частях ринга.
- **Форма зоны:**
круговая зона влияния.
- **Радиус зоны:**
настраиваемый параметр (по умолчанию 1.5–2.5 игровых единиц).
- **Тип воздействия (один или несколько):**
 - притяжение (pull) мяча к центру зоны;
 - искривление траектории (curve) при пролёте через зону.
- **Интенсивность воздействия:**

задаётся коэффициентом силы и применяется постепенно, без резких телепортаций мяча.

7.8.4 Применение магнитного воздействия

При нахождении мяча внутри магнитной зоны сервер на каждом тике:

- изменяет вектор скорости мяча;
- корректирует направление движения в зависимости от типа зоны;
- сохраняет ограничения по минимальной и максимальной скорости мяча.

Все расчёты магнитных зон выполняются **на сервере** в рамках server-authoritative логики.

7.8.5 Визуализация магнитных зон

- Визуальная индикация магнитных зон **может быть минимальной или отсутствовать**.
- Для MVP допускается:
 - полностью логическая зона без визуального отображения;
 - либо простая визуальная подсказка (декор/маркер без эффектов).
- Отсутствие сложной визуализации **не считается дефектом MVP**.

7.8.6 Конфигурация и расширяемость

- Все параметры магнитных зон (радиус, сила, тип) должны быть вынесены в конфигурацию.
- Архитектура должна позволять:
 - изменение количества зон;
 - включение/отключение зон;
 - добавление новых типов воздействия в будущих этапах.

7.8.7 Критерии приёмки (Acceptance Criteria)

Раздел считается реализованным, если:

1. В игре присутствуют минимум 4 магнитные зоны.
2. Мяч меняет траекторию при попадании в зону влияния.
3. Воздействие магнитных зон стабильно и не ломает физику мяча.
4. Магнитные зоны не влияют на движение игроков.
5. Поведение мяча одинаково у всех игроков (серверная физика).

7.9 Тень мяча (blob shadow)

7.9.1 Требование

Под мячом должна отображаться лёгкая “пятнистая” тень:

- круг/эллипс под мячом
- без real-time shadowmap (по возможности)

7.9.2 Реализация

- plane с текстурой/градиентом, привязан к мячу по XZ
- Y фиксирован (почти на полу, например 0.01)
- масштаб тени может слегка меняться от скорости (опционально)

7.10 Синхронизация мяча по сети

7.10.1 Сервер

Сервер отправляет клиентам:

- позицию мяча
- скорость мяча (опционально, для интерполяции)

7.10.2 Клиент

Клиент обязан:

- визуально интерполировать движение мяча между снапшотами;
- не выполнять реальную физику локально;
- корректно отображать резкие изменения направления после удара (без “догоняния” в обратную сторону).

7.11 Критерии приёмки (Acceptance Criteria)

Мяч считается реализованным корректно, если:

1. Мяч движется по плоскости, не подпрыгивает.
2. Мяч отражается от бортов и углов арены без “залипания”.
3. Мяч отражается от игроков, траектория меняется от точки контакта.
4. Движение игрока влияет на силу/направление удара.
5. Скорость мяча изменяется, но ограничена min/max.
6. Под мячом отображается лёгкая тень.
7. Физика выполняется на сервере, у всех игроков состояние совпадает.

8. Сетевая модель и протокол WebSocket

8.1 Назначение сетевого слоя

Сетевой слой обеспечивает:

- подключение клиентов к матчу;
- передачу ввода игроков на сервер;
- передачу серверного состояния (снапшотов) всем клиентам;
- синхронизацию движения игроков и мяча в реальном времени;
- устойчивую работу в условиях задержек, потерь пакетов и реконнектов.

Архитектура строго **server-authoritative**:

клиенты не отправляют позиции и не вычисляют физику мяча.

8.2 Базовые требования к соединению

- Протокол: WebSocket
- Сервер: Node.js + ws
- Клиент: WebSocket API браузера
- Подключение: **один WS на клиента**
- Кодировка сообщений: **JSON** (для MVP)
- В дальнейшем допускается переход на бинарный протокол, но в MVP не требуется.

8.3 Состояния клиента (Client Connection State)

Клиент должен иметь следующие состояния:

1. DISCONNECTED — нет соединения
2. CONNECTING — выполняется подключение
3. CONNECTED — подключение установлено, ожидание допуска в матч
4. IN_MATCH — клиент в активном матче
5. RECONNECTING — попытки восстановить соединение
6. REJECTED — отказ (комната заполнена / ошибка)

В UI отображается минимум:

- текущий статус (Connected / Reconnecting / Rejected)
- ping

8.4 Логика комнаты (Room Rules)

- Комната одна (MVP): roomId = "default"
- Максимум игроков: 4
- Сервер назначает игроку:
 - playerId
 - side (north/south/west/east)
- Если слоты заняты:
 - сервер отправляет error: ROOM_FULL
 - закрывает соединение (или держит без IN_MATCH — по реализации, но MVP: закрыть)

8.5 Частоты и временные параметры

8.5.1 Частота серверного тика

- serverTickRate = 60 Hz (dt = 16.666 ms)

8.5.2 Частота снимков

- snapshotRate = 20–60 Hz (настройка)
- Рекомендуемо для MVP: 30 Hz

(баланс плавности и нагрузки)

8.5.3 Пинг

- Сервер отправляет heartbeat ping каждые 2 секунды
- Клиент отвечает pong
- Клиент отображает round-trip time в HUD

8.6 Общий формат сообщений

Каждое сообщение — JSON-объект с обязательными полями:

```
{
  "type": "string",
  "t": 0,
  "payload": {}
}
```

Где:

- type — тип сообщения
- t — серверное время или tick (в зависимости от направления)
- payload — данные типа сообщения

Для MVP допускается t как serverTick (целое), чтобы клиент мог интерполировать.

8.7 Типы сообщений: Client → Server

8.7.1 HELLO (handshake / вход)

Отправляется сразу после открытия WS.

Назначение: идентификация и передача telegram-данных.

```
{
  "type": "HELLO",
  "payload": {
    "telegramUserId": "123456789",
    "username": "nick",
    "initData": "optional"
  }
}
```

Требования:

- telegramUserId обязателен
- username может быть пустым → сервер делает fallback Player####
- initData (если используется) передаётся как строка

8.7.2 INPUT (ввод игрока)

Отправляется:

- при изменении ввода
- либо фиксированно, например 20–30 раз/сек (предпочтительно для стабильности)

```
{
  "type": "INPUT",
  "payload": {
    "seq": 17,
    "dir": { "x": 0.0, "z": 1.0 },
    "actions": {
      "pause": false
    }
  }
}
```

Где:

- seq — номер команды (монотонно растущий, int)
- dir.x, dir.z ∈ [-1;1]
- actions — набор флагов (в MVP можно оставить как задел)

Правила:

- Диагональ должна быть нормализована на клиенте
- Сервер валидирует диапазоны и отбрасывает мусор

8.7.3 PONG (ответ на ping)

```
{
  "type": "PONG",
  "payload": { "pingId": 3 }
}
```

8.7.4 DEBUG (опционально, только dev)

Например сброс мяча:

```
{
  "type": "DEBUG",
  "payload": { "cmd": "RESET_BALL" }
}
```

Важно: В прод-режиме DEBUG должен быть выключен конфигом.

8.8 Типы сообщений: Server → Client

8.8.1 WELCOME (допуск в матч)

Сервер подтверждает вход и назначает роль.

```
{
  "type": "WELCOME",
  "payload": {
    "playerId": "p1",
    "side": "north",
    "roomId": "default",
    "tickRate": 60,
    "snapshotRate": 30
  }
}
```

8.8.2 ROOM_STATE (список игроков)

Отправляется:

- при подключении/отключении игроков
- при реконнекте

```
{
  "type": "ROOM_STATE",
  "payload": {
    "players": [
      { "playerId": "p1", "username": "nick1", "side": "north" },
      { "playerId": "p2", "username": "nick2", "side": "south" }
    ]
  }
}
```

```
]
}
}
```

8.8.3 SNAPSHOT (состояние игры)

Отправляется регулярно с частотой snapshotRate.

```
{
  "type": "SNAPSHOT",
  "t": 12450,
  "payload": {
    "ball": {
      "pos": { "x": 1.2, "z": -0.4 },
      "vel": { "x": 3.1, "z": 2.2 }
    },
    "players": [
      { "playerId": "p1", "pos": { "x": 0.0, "z": 3.8 } },
      { "playerId": "p2", "pos": { "x": 1.0, "z": -3.6 } }
    ]
  }
}
```

Требования:

- t — serverTick (int)
- players содержит всех активных игроков
- позиция всегда в плоскости XZ

8.8.4 PING (heartbeat)

```
{
  "type": "PING",
  "payload": { "pingId": 3, "ts": 1700000000 }
}
```

8.8.5 ERROR (ошибка)

```
{
  "type": "ERROR",
  "payload": {
    "code": "ROOM_FULL",
    "message": "Room is full"
  }
}
```

Список обязательных кодов ошибок MVP:

- ROOM_FULL
- BAD_HELLO
- BAD_INPUT
- SERVER_ERROR

8.8.6 MATCH_EVENT (события матча)

В MVP события минимальные, но протокол закладывается.

Примеры:

- старт матча
- гол/очко (если правило есть)
- окончание раунда

```
{
  "type": "MATCH_EVENT",
  "payload": {
    "event": "MATCH_START"
  }
}
```

8.9 Валидация и защита от некорректных данных

Сервер обязан:

- ограничивать частоту INPUT от одного клиента (rate limit);
- нормализовать dir при необходимости;
- игнорировать сообщения неизвестных типов;
- не допускать падения сервера от неверного JSON.

Клиент обязан:

- не отправлять INPUT чаще согласованного лимита (например 30 Hz);
- отправлять HELLO строго один раз при подключении.

8.10 Реконнект (Reconnection Policy)

8.10.1 На клиенте

- При разрыве WS:
 - перейти в RECONNECTING
 - попытки переподключения: 1, 2, 3, 5, 8 секунд (экспоненциально)
 - максимум 5 попыток, далее статус “Disconnected”

8.10.2 На сервере

- При реконнекте по telegramUserId:
 - восстановить того же игрока (если слот не занят другим)
 - выдать WELCOME повторно
 - отправить ROOM_STATE
 - продолжить отправку SNAPSHOT

8.11 Критерии приёмки (Acceptance Criteria)

Сетевой протокол считается реализованным, если:

1. 4 клиента подключаются и получают уникальные side.
2. 5-й клиент получает ERROR ROOM_FULL и не попадает в матч.
3. Клиенты отправляют только INPUT, сервер считает позиции и физику.
4. Сервер стабильно отправляет SNAPSHOT, клиенты плавно отображают движение.

5. Пинг отображается и обновляется (heartbeat работает).
6. При разрыве соединения клиент делает реконнект и возвращается в матч (в пределах 5 попыток).
7. Некорректные сообщения не валидировать сервер.

9. Игровые правила, логика матча и состояния игры

9.1 Назначение игровых правил в MVP

Игровые правила в рамках MVP предназначены для:

- формализации начала, хода и завершения матча;
- обеспечения понятного и воспроизводимого игрового процесса;
- возможности тестирования сетевой и физической логики;
- подготовки основы для расширения правил в следующих версиях.

Правила MVP **упрощены** и не содержат сложной логики подсчёта очков, таймеров или режимов, если иное не указано.

9.2 Тип матча

- Тип матча: **Free-for-all (4 игрока, каждый сам за себя)**.
- Количество игроков: **ровно 4**.
- Матч начинается **только при наличии 4 игроков**.
- До заполнения всех слотов матч не стартует.

9.3 Состояния матча (Match State Machine)

Матч должен иметь строго определённые состояния:

1. WAITING_FOR_PLAYERS
2. READY
3. IN_PROGRESS
4. PAUSED (только debug / dev)
5. FINISHED

9.3.1 WAITING_FOR_PLAYERS

- Начальное состояние сервера.
- Сервер принимает подключения.
- Игроки получают свои стороны арены.
- Мяч не активен (скорость = 0).

Условие выхода:

- подключились 4 игрока.

9.3.2 READY

- Короткое подготовительное состояние (1–3 секунды).
- Игроки зафиксированы на стартовых позициях.
- Мяч находится в центре арены.

Назначение:

- дать клиентам время синхронизироваться;
- отобразить визуальный сигнал старта (опционально).

После таймера → IN_PROGRESS.

9.3.3 IN_PROGRESS

- Основное игровое состояние.
- Игроки могут двигаться.
- Мяч активен, физика включена.
- Сервер рассылает снапшоты.

Это состояние длится до выполнения условия завершения матча.

9.3.4 PAUSED (только для MVP/debug)

- Может быть активировано:
 - через debug-команду;
 - либо при потере всех клиентов (опционально).
- Физика мяча останавливается.
- Позиции не обновляются.

В прод-версии может быть отключено.

9.3.5 FINISHED

- Матч завершён.
- Физика мяча остановлена.
- Ввод игроков игнорируется.
- Клиенту отправляется событие завершения.

После этого:

- либо сервер возвращается в WAITING_FOR_PLAYERS;
- либо матч перезапускается (по конфигу).

9.4 Старт матча

9.4.1 Условия старта

Матч стартует, если:

- подключены 4 уникальных игрока;
- все клиенты подтвердили получение WELCOME;
- сервер перешёл в состояние READY.

9.4.2 Стартовые условия

- Все игроки:
 - находятся в центре своей стороны;
 - не могут двигаться в течение READY.
- Мяч:
 - позиция (0, 0, 0);
 - скорость (0, 0).

После старта:

- сервер задаёт мячу начальную скорость baseSpeed

- направление выбирается:
 - либо случайно,
 - либо по заранее заданной логике (например, в сторону случайного игрока).

9.5 Правила взаимодействия с мячом

- Любой игрок может отбить мяч при столкновении.
- Один удар = одно столкновение, обработанное сервером.
- Повторные удары без выхода мяча из зоны контакта блокируются кулдауном (см. п.7).

9.6 Условия завершения матча (MVP)

Для MVP допускается **один из вариантов**, выбираемый на этапе реализации:

Вариант А (рекомендуемый для MVP)

Тайм-матч

- Матч длится фиксированное время (например, 2–3 минуты).
- По истечении времени матч завершается.
- Победитель не определяется (sandbox / тестовый режим).

Вариант В

Очковая система (минимальная)

- Если мяч пересекает сторону игрока и касается борта за ним → игрок получает “пропуск”.
- Пропуски считаются.
- Матч завершается при достижении лимита (например, 5 пропусков у любого игрока).

Для MVP допустимо отсутствие финального победителя — важно, чтобы матч корректно стартовал и завершался.

9.7 Обработка выхода игрока из матча

9.7.1 Выход во время матча

Если игрок покидает матч (disconnect):

- сервер помечает игрока как disconnected;
- его сторона считается пустой;
- матч **прерывается** и переходит в FINISHED
 - либо возвращается в WAITING_FOR_PLAYERS (MVP-упрощение).

9.7.2 Реконнект

Если игрок переподключился:

- в течение допустимого времени (см. п.8);
- и слот не занят;

→ он возвращается на свою сторону, матч продолжается.

9.8 События матча (Match Events)

Сервер обязан отправлять клиентам события:

- MATCH_READY
- MATCH_START
- MATCH_END

Пример:

```
{
  "type": "MATCH_EVENT",
  "payload": {
    "event": "MATCH_START"
  }
}
```

9.9 Отображение правил на клиенте (UI)

В MVP клиент обязан:

- показать визуальный сигнал старта матча (текст / индикатор);
- показать завершение матча;
- заблокировать управление вне IN_PROGRESS.

9.10 Ограничения правил MVP

В MVP:

- отсутствует сложная логика подсчёта очков (если не выбрана явно);
- отсутствует система победителей/рейтингов;
- отсутствует сохранение результатов;
- отсутствует повтор матча (replay).

9.11 Критерии приёмки (Acceptance Criteria)

Игровые правила считаются реализованными, если:

1. Матч не стартует, пока не подключены 4 игрока.
2. Матч проходит через состояния WAITING → READY → IN_PROGRESS → FINISHED.
3. Игроки не могут двигаться до старта.
4. Мяч активируется строго в IN_PROGRESS.
5. Матч корректно завершается по заданному условию.
6. Клиенты получают события старта и завершения.

10. Управление, ввод и пользовательский интерфейс (Controls & UI)

10.1 Назначение раздела

Данный раздел описывает требования к:

- способам управления персонажем;
- обработке пользовательского ввода;
- пользовательскому интерфейсу (UI);
- отображению служебной информации во время матча.

10.2 Общие требования к управлению

- Управление должно быть:
 - интуитивным;
 - отзывчивым;
 - устойчивым к задержкам ввода.
- Управление не должно:
 - требовать точных кликов;
 - перекрывать основную игровую область;
 - приводить к случайным нажатиям.
- Клиент передаёт на сервер **только вектор ввода**, а не позицию.

10.3 Управление на мобильных устройствах (основное)

10.3.1 Тип управления

Для мобильных устройств **обязательно** реализуется один из вариантов (или оба):

- **Экранный стик (виртуальный джойстик)** — предпочтительно
- **D-pad (стрелки)** — допустимо

Выбор конкретной реализации допускается на этапе разработки, но функциональные требования одинаковы.

10.3.2 Экранный стик (если выбран)

Требования:

- располагается в нижней части экрана (обычно слева);
- состоит из:
 - базовой зоны (радиус);
 - подвижного стика;
- реагирует на:
 - касание;
 - удержание;
 - свайп.

Поведение:

- направление движения определяется вектором от центра стика;
- сила движения нормализуется (без ускорения);
- отпускание стика $\rightarrow \text{dir} = (0,0)$.

10.3.3 D-pad (если выбран)

Требования:

- 4 кнопки: вверх / вниз / влево / вправо;
- поддержка одновременных нажатий (диагональ);
- визуально крупные зоны касания.

10.3.4 Параметры тач-ввода

- Минимальный размер зоны касания: **44×44 px**.
- Поддержка multitouch (минимум 2 одновременных касания).
- Корректная работа при свайпах за пределы зоны стика.

10.4 Управление на desktop (вторично)

Поддерживается для отладки и desktop Telegram.

- Клавиши:
 - W — вперёд
 - S — назад
 - A — влево
 - D — вправо
- Альтернатива: стрелки клавиатуры.

Desktop-управление **не является приоритетным**, но должно работать корректно.

10.5 Обработка ввода (Input Pipeline)

10.5.1 На клиенте

- Ввод собирается каждый кадр.
- Формируется вектор:
 - $\text{dir.x} \in [-1, 1]$
 - $\text{dir.z} \in [-1, 1]$
- Диагональ нормализуется:
 - длина вектора не превышает 1.

10.5.2 Отправка на сервер

- INPUT отправляется:
 - при изменении направления;
 - либо с фиксированной частотой (20–30 Hz).
- Клиент не должен:
 - отправлять INPUT каждый кадр без необходимости;
 - превышать лимиты частоты.

10.6 Блокировка управления

Управление должно быть **заблокировано**, если:

- матч не в состоянии IN_PROGRESS;
- клиент находится в состоянии DISCONNECTED;
- матч завершён (FINISHED).

В этих состояниях:

- ввод не отправляется на сервер;
- визуально управление может быть приглушено (opacity).

10.7 Пользовательский интерфейс (UI)

10.7.1 Общие требования

- UI минималистичный;

- не перекрывает важные игровые объекты;
- адаптирован под разные размеры экранов;
- учитывает safe-area (вырезы, закругления).

10.8 HUD (Heads-Up Display)

10.8.1 Обязательные элементы HUD

1. Сетевой статус

- Connected / Reconnecting / Disconnected
- цветовая индикация допустима

2. Ping

- отображается в миллисекундах
- обновляется раз в несколько секунд

3. Список игроков

- никнейм
- сторона арены (иконка или текст)

4. Подсказки управления

- краткие
- могут скрываться после начала матча

10.8.2 Никнеймы над игроками

- Отображаются над моделью игрока.
- Реализация:
 - billboard (Sprite / CSS2DObject);
- Никнейм всегда повернут к камере.
- Никнейм не должен сильно перекрывать других игроков.

10.9 Debug-элементы (только для MVP)

Допускаются следующие debug-функции:

- Пауза матча (Space);
- Сброс мяча (R);
- Кнопки Connect / Disconnect (скрываемые).

Требование:

Все debug-элементы должны быть:

- легко отключаемы конфигом;
- не попадать в финальную прод-сборку без необходимости.

10.10 Обработка ошибок в UI

При возникновении ошибок:

- потеря соединения;
- отказ входа в комнату;

- критическая ошибка сервера;

Клиент должен:

- отобразить понятное сообщение;
- предложить переподключение (если возможно);
- не зависать в неконсистентном состоянии.

10.11 Критерии приёмки (Acceptance Criteria)

UI и управление считаются реализованными, если:

1. Управление работает на мобильных устройствах без лагов.
2. Игрок может непрерывно двигаться удержанием стика/кнопки.
3. Ввод корректно блокируется вне IN_PROGRESS.
4. HUD отображает статус соединения и ping.
5. Никнеймы отображаются над персонажами.
6. UI не перекрывает важные части сцены и не ломается при resize.

11. Производительность, оптимизация и ограничения MVP

11.1 Назначение раздела

Данный раздел определяет **жёсткие требования к производительности**, обязательные для MVP, с учётом запуска игры внутри **Telegram Mini Apps (WebView)** на мобильных устройствах.

Производительность является **критическим фактором приёмки**: визуально корректная, но нестабильная игра считается **неприемлемой**.

11.2 Целевые показатели производительности

11.2.1 FPS

- Целевой средний FPS: ≥ 45
- Желаемый FPS: **50–60**
- Допустимые кратковременные просадки:
 - не ниже 30 FPS
 - не дольше 200–300 мс

11.2.2 Время кадра

- Среднее время кадра: ≤ 20 мс
- Пики CPU/GPU не должны приводить к фризам.

11.3 Ограничения по графике (обязательные)

11.3.1 Освещение

- Максимум:
 - 1 DirectionalLight
 - 1 AmbientLight
- Динамическое изменение света запрещено.
- ShadowMap:
 - либо отключены полностью,
 - либо используются **только для одного источника** и с минимальным разрешением.

11.3.2 Тени

- Предпочтительный вариант:
 - **blob-shadow** (проекционная тень) для мяча и персонажей.
- Запрещено:
 - real-time мягкие тени;
 - каскадные shadow maps;
 - множественные источники теней.

11.3.3 Материалы и шейдеры

- Использовать стандартные материалы Three.js:
 - MeshStandardMaterial (упрощённый),
 - либо MeshLambertMaterial.
- Запрещено:
 - кастомные GLSL-шейдеры;
 - сложные PBR-настройки;
 - отражения и преломления.

11.4 Ограничения по ассетам

11.4.1 Геометрия

- Низко- или средне-полигональные модели.
- Отсутствие:
 - скрытых мешей;
 - задников;
 - декоративных деталей вне зоны обзора.

11.4.2 Текстуры

- Текстуры по возможности не использовать.
- Если используются:
 - разрешение $\leq 512 \times 512$;
 - формат сжатия (при возможности);
 - минимум альфа-каналов.

11.5 Ограничения по сцене

- Сцена полностью статична (кроме игроков и мяча).
- Отсутствуют:
 - частицы;
 - анимации окружения;
 - пост-эффекты.
- Количество draw calls должно быть минимальным:
 - объединение мешей допускается.

11.6 Оптимизация клиентского кода

11.6.1 Рендер-цикл

- Использовать один requestAnimationFrame.
- Не выполнять тяжёлые вычисления в каждом кадре.
- Вынести математику физики на сервер.

11.6.2 Обработка resize

- Resize не должен:
 - пересоздавать сцену;

- пересоздавать ассеты;
- приводить к утечкам памяти.

11.7 Оптимизация сетевого взаимодействия

- Ограничить частоту INPUT сообщений (20–30 Hz).
- Ограничить размер SNAPSHOT сообщений.
- Исключить передачу избыточных данных (например, rotation = 0).

11.8 Ограничения серверной части

- Сервер обязан:
 - выдерживать стабильный tick 60 Hz;
 - не блокироваться синхронными операциями;
 - корректно обрабатывать отключения клиентов.
- Один сервер = одна игровая комната (MVP).

11.9 Ограничения памяти

- Клиент не должен:
 - удерживать старые снапшоты бесконечно;
 - создавать новые объекты Three.js каждый кадр;
 - накапливать обработчики событий.

11.10 Поведение при деградации производительности

Если производительность падает:

- приоритет отдаётся:
 1. стабильности управления;
 2. корректности физики;
 3. визуальному качеству.
- допустимо:
 - отключение теней;
 - снижение частоты снапшотов;
 - упрощение UI (в MVP — ручное, не автоматическое).

11.11 Критерии приёмки (Acceptance Criteria)

Производительность считается соответствующей требованиям, если:

1. Игра стабильно работает в Telegram WebView на iOS и Android.
2. Средний FPS ≥ 45 в матче на 4 игроков.
3. Нет регулярных фризов и “стоп-кадров”.
4. Управление остаётся отзывчивым при нагрузке.
5. Нет утечек памяти при длительной сессии (5–10 минут).

12. Тестирование и сценарии проверки MVP

12.1 Назначение тестирования

Тестирование необходимо для подтверждения того, что:

- MVP реализован строго в соответствии с ТЗ;
- игра стабильно работает в среде Telegram Mini Apps;
- отсутствуют критические ошибки, делающие игру неиграбельной;
- сетевое взаимодействие и физика корректны при реальных условиях.

Тестирование проводится до передачи результата заказчику и является обязательной частью работ.

12.2 Уровни тестирования

Тестирование MVP проводится на следующих уровнях:

1. Локальное функциональное тестирование (без сети)
2. Локальное сетевое тестирование
3. Тестирование в Telegram WebView
4. Стресс-тестирование (в рамках MVP)

12.3 Локальное функциональное тестирование (Offline)

12.3.1 Цель

Проверка:

- базовой логики сцены;
- управления персонажем;
- физики мяча без сетевого слоя.

12.3.2 Обязательные проверки

- Сцена корректно загружается.
- Арена соответствует размерам и координатам.
- Персонаж:
 - появляется на корректной стороне;
 - двигается по XZ;
 - не вращается;
 - не выходит за границы зоны.
- Мяч:
 - появляется в центре;
 - движется по плоскости;
 - отражается от бортов;
 - не “залипает” в углах.

12.4 Локальное сетевое тестирование

12.4.1 Цель

Проверка:

- WebSocket-протокола;
- server-authoritative логики;
- синхронизации между клиентами.

12.4.2 Сценарии

1. Запуск сервера локально.
2. Подключение 4 клиентов (4 вкладки / браузера).
3. Проверка:
 - назначение сторон;
 - синхронное движение игроков;
 - одинаковое поведение мяча у всех клиентов.
4. Подключение 5-го клиента:
 - получение ошибки ROOM_FULL;

- отсутствие попадания в матч.

12.5 Тестирование реконнекта

12.5.1 Сценарии

- Отключить интернет на одном клиенте.
- Проверить:
 - переход в состояние RECONNECTING;
 - попытки переподключения.
- Включить интернет.
- Проверить:
 - восстановление соединения;
 - возврат игрока на свою сторону;
 - продолжение матча (если допустимо по правилам).

12.6 Тестирование в Telegram Mini Apps

12.6.1 Среда тестирования

- Telegram iOS (реальное устройство).
- Telegram Android (реальное устройство).
- Telegram Desktop (дополнительно).

12.6.2 Обязательные проверки

- Игра запускается внутри WebView.
- Нет ошибок и предупреждений в консоли.
- Корректно определяется размер экрана.
- Управление работает через тач:
 - без “пропаданий” ввода;
 - без случайных кликов.
- UI не перекрывается системными элементами Telegram.
- Нет горизонтального скролла страницы.

12.7 Производительное тестирование

12.7.1 Сценарии

- Матч на 4 игроков длительностью ≥ 5 минут.
- Проверка:
 - стабильности FPS;
 - отсутствия накопления лагов;
 - отсутствия деградации управления.

12.7.2 Критические признаки проблем

- регулярные фризы;
- постепенное падение FPS;
- рассинхронизация мяча;
- “телепортация” игроков.

Наличие любого из этих признаков требует исправления.

12.8 Тестирование граничных сценариев

Обязательно проверить:

- быстрые смены направления движения;
- удары мяча под острым углом;
- попадания мяча в углы;
- одновременные удары двух игроков (почти одновременно);
- выход игрока в момент удара.

12.9 Документирование результатов тестирования

Исполнитель обязан:

- зафиксировать найденные критические баги;
- устранить их до передачи MVP;
- повторно проверить сценарии после исправлений.

12.10 Критерии приёмки тестирования

MVP считается прошедшим тестирование, если:

1. Все обязательные сценарии выполнены без критических ошибок.
2. Игра стабильно работает в Telegram WebView.
3. Сетевая синхронизация корректна.
4. Управление отзывчиво.
5. Производительность соответствует требованиям (см. п.11).

13. Документация, передача результата и критерии завершения работ (Definition of Done)

13.1 Назначение раздела

Данный раздел фиксирует:

- какие материалы и артефакты должны быть переданы заказчику;
- в каком виде считается, что работа выполнена;
- по каким критериям MVP принимается как завершённый.

Этот раздел является **финальной точкой ТЗ** и используется для формальной приёмки результата.

13.2 Передаваемые материалы (Deliverables)

Исполнитель обязан передать заказчику следующий комплект материалов:

13.2.1 Исходный код

1. Клиентская часть

- исходный код Next.js-проекта;
- Three.js сцена и логика;
- UI и управление;
- конфигурационные файлы.

2. Серверная часть

- исходный код Node.js WebSocket-сервера;
- конфигурация портов, частот, параметров физики;
- скрипт запуска сервера.

Код должен:

- быть читаемым;
- иметь базовую структуру;
- не содержать временных “заглушек”, влияющих на MVP.

13.2.2 Активы

- Все используемые GLB-файлы:
 - arena.glb
 - player.glb (или per-character)
 - ball.glb
- Активы должны соответствовать требованиям ТЗ:
 - корректный pivot;
 - корректный масштаб;
 - оптимизированный вес.

13.2.3 Сборка клиента

- Production-сборка клиента, готовая к запуску в Telegram Mini Apps.
- Клиент должен запускаться без дополнительных правок кода.

13.3 Документация (обязательная)

13.3.1 README.md

README должен содержать:

1. Краткое описание проекта.
2. Требования к среде:
 - Node.js версия;
 - npm / yarn / pnpm.
3. Инструкцию по запуску сервера:
 - установка зависимостей;
 - запуск WS-сервера;
 - конфигурация порта.
4. Инструкцию по запуску клиента:
 - установка зависимостей;
 - запуск Next.js;
 - production build.
5. Перечень конфигурационных параметров:
 - скорости;
 - частоты;
 - лимиты игроков.
6. Описание структуры проекта (кратко).

13.3.2 Инструкция по деплою

Отдельный раздел или файл должен описывать:

- как развернуть WebSocket-сервер на хостинге;
- какой URL использовать клиенту;
- какие порты должны быть открыты;
- как проверить, что сервер работает.

13.4 Требования к качеству результата

Результат считается качественным, если:

- код не содержит критических багов;
- отсутствуют падения клиента или сервера;
- управление и физика работают стабильно;
- игра не “ломается” при типовых действиях пользователя;
- производительность соответствует требованиям раздела 11.

13.5 Definition of Done (DoD)

MVP считается **полностью завершённым**, если выполнены все пункты ниже:

1. Игра запускается внутри Telegram Mini Apps без ошибок.
2. 4 игрока могут подключиться к одному матчу.
3. 5-й игрок получает отказ (ROOM_FULL).
4. Матч проходит все состояния:
 - WAITING → READY → IN_PROGRESS → FINISHED.
5. Персонажи:
 - двигаются корректно;
 - не вращаются;
 - ограничены своей стороной.
6. Мяч:
 - движется по плоскости;
 - корректно отражается от игроков и стен;
 - не застревает в углах.
7. Физика выполняется на сервере.
8. Клиенты получают и корректно интерполируют снапшоты.
9. Управление работает на мобильных устройствах.
10. FPS соответствует требованиям MVP.
11. Переданы исходники, ассеты и документация.

13.6 Основания для отказа в приёме

Заказчик вправе не принять работу, если:

- игра не запускается в Telegram WebView;
- невозможно подключить 4 игроков;
- присутствуют критические баги (краш, зависание, рассинхрон);
- физика мяча не соответствует описанию ТЗ;
- управление неработоспособно на мобильных устройствах;
- отсутствует документация или сборка.

13.7 Завершение работ

После выполнения всех условий:

- MVP считается сданным;
- дальнейшие улучшения, расширения и новые режимы

оформляются **как отдельный этап разработки**.